Trips — design

Date: 2026-10-13 · Status: design approved in chat (parts A, B, C and the nav tab); revised after a 3-lens red-team review (1 blocker, ~14 majors fixed) and verification rounds 2–3; written spec awaiting Alex's review · Phase: 18 (builds after Phase 17, subscription marking)

Goal

While on holiday, Alex wants a set budget per category (food, shopping, hotels) and per city, and to see at a glance how he is doing. Today he stars holiday spending as "special" and can file it in a special-expense group ("September 2026 Paris holiday"). That keeps it out of his monthly budgets, but gives him no trip budget, no cities and no dates, and costs about 8 taps per expense abroad, because Quick Add forgets the group and currency every time it opens. (Verified in code, 2026-10-13.)

Decisions (Alex, 2026-10-13)

Question	Decision
Relationship to monthly budgets	Trips are their own thing, with a per-trip switch: "Keep this trip out of my monthly budget".
Cities	Stops with dates. Trim picks the city from the expense's date; it can be changed on any expense.
Budget lines	An overall trip budget + per-category budgets for the whole trip + one total per city.
Encryption	Encrypt trip names, city names and every trip budget, like the rest of the money data.
Existing holiday groups	"Start from a special group" converts a group into a trip.
Navigation	Its own "Trips" tab.
Budget currency	Home currency (stated as an assumption in chat; not objected to).

Two rules that shape everything

1. Membership is stored; the city is derived. An expense belongs to a trip because it carries `trip_id`. Its city is worked out from its date (`stopForDate`) every time figures are read. `trip_stop_id` is stored only as an explicit override, when Alex picks a different city than the date implies. This follows the same "derived, never stored" reasoning as Phase 17, and it removes a whole class of stale data: - correcting a stop's dates re-files that stop's expenses automatically; - re-dating an expense moves it to the right city; - bulk-adding and converting a group need no per-row city writes; - moving an expense to another trip cannot leave behind a city from the old trip.

Pre-trip bookings (flights) are dated outside every stop, so they land under "No city" by themselves.

2. Only the trip decides the star. The trip owns `is_special` while it is attached: - Quick Add does not send `isSpecial` while a trip is attached; the server derives it. (Red-team blocker: Quick Add

always sent `isSpecial: false`, so every 3-tap trip expense would have counted in the monthly budget.) - The Edit dialog sends `isSpecial` (and `tripId`) only when Alex changed it in that visit. - The server derives the star only when the trip actually changes. - The row star stays a deliberate per-expense override, sent explicitly.

What Alex will see

Trips page (**`/trips`**, new nav tab)

- Sections: On now, Coming up, Past. Past includes archived trips, labelled “Archived”, each with Unarchive.
- Each card shows name, dates, and “£640 left · £128/day”, or “Starts in 4 days”, or for a finished trip “Came in £120 under [party icon]” / “Came in £55 over — worth a bigger budget next time?” (amber, never red).
- New trip button. Start from a special group button (only when Alex has groups).
- A Quick Add button (FAB), as on Dashboard and Transactions.
- Empty state: “Planning a getaway? Give it a budget before you go ✈”.

Setting up a trip (trip editor — one save for everything)

- Name. Start and end dates (required, entered by Alex). Optional overall budget (`MoneyInput`, home currency).
- Switch: “Keep this trip out of my monthly budget”, on by default. Helper: “Trip spending won’t touch your monthly budgets or pace. It’s tracked here instead.” If the Special-expenses setting is off, the editor says before saving: “This also turns on Special expenses for your account. Anything you starred before will stay out of your budgets too.”
- Stops: city, from–to dates (inside the trip; consecutive stops may share a travel day), optional budget, and optional local currency from the 31 currencies Quick Add can price. VND, which the rate service does not quote, is not offered for stops in v1, and the editor says so.
- Category budgets: an expense category plus an amount, once per category. “+ New category” opens the existing category dialog, exported for this with `lockType="expense"` and an `onCreated` callback that selects the new category. Hidden in simple mode, which files everything under “Other”.
- Trip dates, stops and category budgets are validated together and saved in one request. So postponing a trip by a week, stops included, just works. (Red team: separate saves made this impossible in either order.)

One trip (**`/trips/:id`**)

- Hero: “Day 3 of 8” (or “Starts in 4 days” / “Ended 2 Oct”), a big “£640 left” of the overall budget, and “£128/day for the 5 days left”. Over budget, in amber: “£55 over — nothing left for the last 2 days”.
  - The per-day figure is simply what is left divided by the days left (including today). It is deliberately not the monthly pace comparison, because pre-paid flights and hotels would make every trip read “over pace” on day 1.
  - With no overall budget: total spent, plus the category and city bars.
- By category: a bar per category budget (the same amber→rose tones as Budgets, never red), then “Other categories” with no bar. Hidden in simple mode.

- By city: a bar per stop that has a budget (spent shown for all stops), then “No city”.
- Expenses on this trip: date, note, category, city chip (derived, or “[pin icon] set by you”), amount, and the original currency when foreign. Tapping a row opens the same edit dialog as Transactions (extracted into a shared component, below).
- Add expenses: expenses with no trip and no special group, in two groups: “During the trip” (dated inside it) and “Before the trip” (up to 180 days before the start, for flights and hotels booked ahead), searchable, each with a checkbox.
- A Quick Add button. Opened from this page, it is pre-attached to this trip whatever the date, with the visible tag and ×. The preset counts as a manual choice: changing the date (e.g. to the March flight’s date) never clears it; only × or the chooser does. That is the easy path for pre-bookings and late charges.
- Edit trip, Archive, Delete.
  - Archive hides the trip from Quick Add’s auto-tag and pickers. It keeps its expenses and figures, and is reversible.
  - Delete asks what to do with its expenses: “Count them in my monthly budget again” (un-star) or “Keep them out”. For a trip made from a special group there is also “Put them back in Paris holiday”, which unarchives the group and restores the expenses to it.
- The first time a finished trip is viewed, a trip that came in under budget gets a small celebration (confetti and a toast). This is a once-only, per-device flag.

Trip mode in Quick Add

- When the dialog’s date falls inside a non-archived trip, Quick Add shows a visible tag at the top, “✈ Italy · Rome ×”, and pre-selects the stop’s currency if it has one.
  - × logs this one as a normal expense.
  - Tapping the tag opens a chooser: other stops (an explicit override) or “Not part of this trip”.
- The default path stays amount → category → category: 3 taps, no extra steps.
- The date drives it: back-dated entries use the trip for their date, and a date changed by the field or by the “type it instead” parser re-evaluates the tag. The tag stays as it is once Alex has chosen manually.
- On overlapping dates, a total order decides, the same on server and client: latest start, then shortest span, then earliest created, then id. A day trip (“Pompeii 12–12”) inside “Rome 12–15” therefore wins on the 12th.
- Income: the tag hides and nothing trip-related is sent while Income is selected (refunds are out of scope). It comes back on switching to Expense.
- While a trip is attached: the Special checkbox and the special-group picker hide; neither **`isSpecial`** nor **`specialGroupId`** is sent (the server derives the star and clears any group). The toast reads “Logged to Italy ✈”, with a “Not trip spending” action that takes it off the trip and un-stars it in one tap (for the home phone bill caught by trip mode).
- Never stale, never slow. `['trips']` is fetched when the app starts (with a staleTime), not only when Quick Add opens.
  - Seeding on open happens inside the existing reset effect, computed from `initialDate` and the freshly reset values, not in a second effect. Two effects in one commit share a render’s closure, so a second effect would read the previous session’s amount and flags. A separate effect handles later changes only (trip data arriving, the date changing). It lists the amount and the touched flags in its dependencies and acts only while the tag is untouched.

  - The reset clears the "tag touched" and "currency touched" flags on every open, so a × applies to one entry, not the rest of the session. (Round-2 finding: keyed only on data and date, the dialog stays mounted between opens, so a second expense logged from the same page would be untagged and in pounds.)
  - It never changes a currency Alex picked, and never changes the currency under an amount already typed.
  - On the very first load only, while trips are still loading, the category chips wait with "Checking your trips…". A refetch never resets typed fields.
- The stop's currency, including on day 1: when the tag attaches with a stop currency, that currency is selected immediately.
  - If the device remembers a rate for the pair from the last 30 days, it is used at once, labelled with its full date ("rate from 12 Oct 2026"), while a fresh rate is fetched in the background. If that background refresh fails, the remembered rate stays; the currency never changes.
  - With no remembered rate, the existing foreign-entry gate applies: the chips stay disabled with "Getting today's € rate…" until the rate arrives. That is today's behaviour for any foreign entry, and happens once per currency per device.
  - If the fetch fails, Quick Add uses the existing "rate unavailable" path: the currency stays, Alex types the rate, and the chips stay gated until it is valid. It switches back to the home currency only while no amount is typed and no chip is armed. Switching under a typed amount would turn 4,500 forints into £4,500, the Phase 12 bug class (round 3).
- Outside trip dates: the details section has a Trip picker listing non-archived trips.
- Simple mode: the tag still applies and is sent (both submit paths).

Elsewhere

- Dashboard: a trip card under the hero for each trip that is on now or starts within 7 days (on-now first, at most two). It shows the day or countdown, "£X left · £Y/day", and links to the trip.
- Dashboard Special-expenses panel: a line "£640 of this month's special spend is on trips → Trips", so the hero chip, the panel and the trip card can always be reconciled. Archived groups with no expenses left (i.e. converted ones) are hidden. The empty-state copy stops telling Alex to group trip spending, and points at Trips instead.
- Budgets page and pace line: while a keep-out trip is on now, one muted line: "Italy spending is tracked on its trip →". This explains why monthly Food looks quiet.
- Transactions: rows show "✈ Italy · Rome". The edit dialog is extracted to `components/TransactionEditDialog.jsx` (including Phase 17's Subscription section) and shared with TripDetail. It gains the Trip + stop picker:
  - the dialog sends **`tripId`** only when Alex changed the trip, and **`isSpecial`** only when Alex changed the box. Re-saving a typo fix can then never re-derive (and silently flip) a hand-set star;
  - picking a trip clears the special group in the dialog state and hides the group picker while a trip is selected. The Special box shows what the server will do (ticked for a keep-out trip). "Changed" means touched in this visit;
  - clearing a keep-out trip un-ticks Special;
  - the picker never changes the expense's currency (the dialog's rate logic only fetches when Alex changes currency, so seeding one there would store a foreign amount at rate 1 — the Phase 12 bug class);

  - the picker always includes the row's current trip, even when archived.
- Start from a special group: choose a group, and the trip editor opens with its name filled in and the dates left empty for Alex to enter. A note gives the group's expense date range and how many expenses fall outside the chosen dates; they still join the trip, because membership is by tag. (Red team: pre-filling from the first and last expense included flights booked months earlier, so the "trip" looked on now and auto-tagged everyday spending.) Saving moves the group's expenses onto the trip, sets their star from the switch, and archives the group. The trip remembers `source_group_id`, which is what makes "Put them back" possible.
- Settings: switching Special expenses off while keep-out trips exist shows one line explaining that trip spending would count in monthly budgets again.
- Phase 17 interplay: if the subscription prompt ("Is this your Netflix subscription?") is answered Yes for a trip-tagged charge, attaching it also takes it off the trip and un-stars it, since a subscription is home spending. The toast says so.

How money is counted

- Trip figures count every expense on the trip, starred or not, including pre-trip bookings.
- Keep-out on: an expense joining the trip gets `is_special = true`, through the one existing exclusion rule (`lib/special.js`). Monthly budgets, the pace line, projections, affordability, wins and Analytics' "excl. special" chip therefore need no change.
- Keep-out off: an expense joining the trip keeps its star exactly as it was, so it counts in both the trip and the monthly budget. This is the same for Quick Add, Edit and bulk-add. Two deliberate exceptions:
  - Conversion from a group sets every moved expense's star from the switch. With keep-out off, the editor says before saving: "These 23 expenses will count in their months' budgets again."
  - Moving from a keep-out trip to a keep-out-off one un-stars, because it is "leaving" the keep-out trip.
- Leaving a trip (trip cleared, or × / "Not trip spending"): if the old trip was keep-out and no explicit `isSpecial` was sent, the star is cleared. This is the mirror image of joining.
- Override one expense: the row star still works, and un-starring keeps the expense on the trip.
- Flipping the switch later asks "Update 23 expenses? This also resets any you starred or un-starred by hand." The server re-applies only when the value actually differs from the stored one, and it updates the transactions first, then the trip row. A failure after the first step leaves the stored value unchanged, so retrying re-applies; an unrelated edit (a rename) never touches stars.
- The setting dependency: stars count only while Settings → Special expenses is on. Saving a keep-out trip turns it on after the disclosure above.
- A trip OR a special group, never both — enforced by a database CHECK, and by the server clearing the group whenever a trip is set.
- Income can't be on a trip (database CHECK).
- Home currency throughout. `transactions.amount` is already converted at entry; no trip figure sums `original_amount`.
- "Repeat this" children created by the nightly job are not trip-tagged (existing behaviour, as for special flags).

Data model

New migration **`server/migrations/017b_trips.sql`**, sorting after `017a_subscription_tracking` and before `018`. Live state verified 2026-10-13: 014–017 applied; 012, 018, 018a and 019 not applied. This is re-checked read-only as rollout step 1, immediately before 017b is applied. Each table follows the `015_special_groups.sql` pattern: its own `user_id`, an index, RLS on, four `own_*` policies, and a `grant all … to service_role`.

```sql
create table if not exists public.trips (
  id                    uuid primary key default gen_random_uuid(),
  user_id               uuid not null references auth.users(id) on delete cascade,
  name                  text not null,
  start_date            date not null,
  end_date              date not null,
  total_budget          numeric(14,2) check (total_budget > 0),
  keep_out_of_monthly   boolean not null default true,
  source_group_id       uuid references public.special_groups(id) on delete set null,
  archived_at           timestamptz,
  created_at            timestamptz not null default now(),
  constraint trips_dates_ordered check (end_date >= start_date)
);

create table if not exists public.trip_stops (
  id          uuid primary key default gen_random_uuid(),
  user_id     uuid not null references auth.users(id) on delete cascade,
  trip_id     uuid not null references public.trips(id) on delete cascade,
  name        text not null,
  start_date  date not null,
  end_date    date not null,
  budget      numeric(14,2) check (budget > 0),
  currency    text check (currency ~ '^[A-Z]{3}$'),
  created_at  timestamptz not null default now(),
  constraint trip_stops_dates_ordered check (end_date >= start_date)
);

create table if not exists public.trip_category_budgets (
  id            uuid primary key default gen_random_uuid(),
  user_id       uuid not null references auth.users(id) on delete cascade,
  trip_id       uuid not null references public.trips(id) on delete cascade,
  category_id   uuid not null references public.categories(id) on delete cascade,
  amount_limit  numeric(14,2) not null check (amount_limit > 0),
  created_at    timestamptz not null default now(),
  constraint trip_category_budgets_unique unique (trip_id, category_id)
);

alter table public.transactions
  add column if not exists trip_id uuid references public.trips(id) on delete set null,
  -- An explicit city OVERRIDE only. NULL means "derive the city from the date".
  add column if not exists trip_stop_id uuid references public.trip_stops(id) on delete
```

```
-- Single-row rules get a database backstop (the service role bypasses RLS, and the tes
-- harness enforces no constraints). No existing row has a trip_id, so both hold on cre
alter table public.transactions drop constraint if exists transactions_trip_xor_group;
alter table public.transactions
  add constraint transactions_trip_xor_group check (trip_id is null or special_group_id
alter table public.transactions drop constraint if exists transactions_trip_expense_onl
alter table public.transactions
  add constraint transactions_trip_expense_only check (trip_id is null or type = 'exper
-- plus partial indexes on transactions(trip_id) / (trip_stop_id) where not null
```

- Application-level rules (cross-row, so SQL `CHECK` cannot express them): stop dates inside trip dates; an override stop belongs to the transaction's trip; a stop override requires a trip. A `CHECK (trip_stop_id is null or trip_id is not null)` was considered and rejected: the ordering of two `on delete set null` actions during an account-deletion cascade is not something to bet a migration on.
- The `> 0` checks sit on the plaintext columns until 019 drops them, exactly as for `budgets.amount_limit`. They are added to 019's "can no longer enforce" list, and Zod enforces them at every phase.

Encryption (Phase 9.5 integration)

Registered in `lib/encryptedFields.js`:

Field	kind	notNull
`trips.name`	text	✓
`trips.total_budget`	amount	
`trip_stops.name`	text	✓
`trip_stops.budget`	amount	
`trip_category_budgets.amount_limit`	amount	✓

The deliberately-plaintext list gains: dates, `keep_out_of_monthly`, `trip_stops.currency` (a public code, like `transactions.fx_rate`), and the FK ids.

Because 018, 018a and 019 are not applied to the live database, they are edited in place: - 018 adds the five `_enc` columns (`add column if not exists … text`). It sorts after 017b, so the tables exist on replay. - 018a adds `trips`, `trip_stops` and `trip_category_budgets` to the `guarded` array. - 019 drops the five plaintext columns and `set not null`s the three notNull `_enc` columns. - Unenforced lists, updated by hand: `test/writeBarrier.pg.test.js` GUARDED and `scripts/snapshot-data.mjs` TABLES. - **`docs/PHASE-9.5-PART-A-RUNBOOK.md`**: its hard-coded expected counts move from 17 encrypted columns and 10 barrier triggers to 22 and 13, with the three new tables named. It also notes that 017a and 017b are applied before 018. Otherwise the runbook would tell Alex to stop on a "mismatch". - SECURITY.md "What is encrypted" gains the five fields. - If the live check ever shows 018 or 018a applied, stop and re-plan (re-apply the edited 018 and 018a after 017b). The in-place edit is only safe while they are unapplied.

Codec rules specific to this feature: - Array inserts use `rows.map((r) => encodeWrite(table, uid, r))`, because `encodeWrite` is per-row. `encodeWrite` gains a guard that throws on an array,

so the mistake cannot pass silently in any phase. - There are no new blind indexes. Nothing looks a trip or city up by name in SQL, and all trip arithmetic happens in JS after `decodeRows`. - Known gap found by the review (pre-existing, filed separately): at phase `dual`, the backfill treats a row as done once its first `_enc` column is set. A partial-field PATCH on an older row (e.g. renaming a trip) could therefore leave a second field unencrypted, and the backfill would skip it forever. Trips' PATCH writes every encrypted field of a trip row whenever it writes any, so it does not depend on that fix.

This touches the Phase 9.5 migrations, so it gets an independent review before merge (the `code-reviewer` and `plan-verifier` agents; Codex is unavailable for security work per the hand-off).

The trip figures — one home

New pure module **`server/lib/trips.js`** (no database import, unit-tested). Every route and devMock use it:

- `tripStatus(trip, today)` → `'upcoming' | 'active' | 'past'`, plus `daysUntilStart`, `dayOfTrip`, `length` and `daysLeft`, inclusive. `today` = `dayInZone(await userTimeZone(uid))`.
- `stopForDate(stops, date)` and `tripForDate(trips, date)` use the total order: latest start, then shortest span, then earliest `created_at`, then id. Archived trips are excluded.
- `cityOf(tx, stops)` = the override stop if `trip_stop_id` is set, else `stopForDate(stops, tx.date)`, else null ("No city").
- `summariseTrip(trip, stops, categoryBudgets, txs, today)` → `{ spent, totalBudget, remaining, perDayLeft, overBy, byCategory: [{ categoryId, limit|null, spent }], otherCategoriesSpent, byStop: [{ stopId, budget|null, spent }], noCitySpent, count }`. City totals always add up to `spent`. `perDayLeft` is present only while active (floored at 0; `overBy` carries the excess).

The client mirrors `stopForDate`, `tripForDate` and `cityOf` in `client/src/lib/trips.js`, used for the tag and the row chips. The mirror's comment names the server file as the source of truth, and both state the total order.

API

New router **`server/routes/trips.js`**, mounted `app.use('/api/trips', requireAuth, tripsRouter)` and added to `routeMountTable.test.js` MOUNTS. Mandatory conventions: - every query is `.eq('user_id', req.user.id)`; - `selectFor` + `decodeRow(s)` before reading names or amounts; `encodeWrite` (per row) for writes; responses built field by field; - `.select(...)` on every write whose result is read; - the UUID path guard runs first; - Zod everywhere: names trimmed 1–60 characters; amounts `.positive().finite().max(1e9)`; dates `YYYY-MM-DD`; ids `.uuid()`; stop currency in the quotable list or the user's base; - 400 `{ error, details }` / 404 / 409.

Endpoint	Does
`GET /api/trips`	All trips (name, dates, switch, archived, stops, category budgets, status) with their headline figures (`spent`, `remaining`, `perDayLeft`, `overBy`; for past trips the final under/over). These come from one lean transactions query (`trip_id, amount, date, category_id, trip_stop_id` where `trip_id is not null`, expenses only), paged the Phase 17 way (order by `date` then `id`, advance by the rows each page returned, stop on an empty page, de-duplicate by id), so the database's row cap can never truncate it silently, and no per-trip calls are needed. Feeds the Trips page, the Dashboard card and Quick Add.
`GET /api/trips/:id`	One trip, its full `summariseTrip`, and its expenses as full transaction rows (`presentRows` with `TX_LIST_COLUMNS`, which the shared edit dialog needs). Paged the same Phase 17 way.
`POST /api/trips`	Body `{ name, startDate, endDate, totalBudget?, keepOutOfMonthly, stops[], categoryBudgets[] }`. The whole body is validated first: stops inside the dates, categories the user's expense categories, no duplicate category (a JS check, then 409). Then it inserts the trip, then the children (array-mapped `encodeWrite`). If a child insert fails, the trip is deleted (best effort; children cascade). For keep-out with the setting off, it turns the setting on and returns `specialEnabledNow: true`.

Endpoint	Does
`PATCH /api/trips/:id`	The editor's single save. Optional `{ name, startDate, endDate, totalBudget (null clears), keepOutOfMonthly, archived, stops[], categoryBudgets[] }`. The lists, when present, are the full desired lists, validated together with the new dates. Stops diff by id: an id must belong to this trip, no id means new, a missing id is deleted (after nulling overrides that point at it), so tagged expenses keep their stop identity. Category budgets diff by **`category_id`**, not row id: categories no longer present are deleted, kept categories get their amount updated, new categories are inserted. Swapping two lines is then just two amount updates, and the unique constraint can never trip midway. Apply order: validate everything → deletes → updates → inserts. The keep-out re-apply runs as described in "How money is counted".
`DELETE /api/trips/:id?expenses=count|keep|restore`	Each mode handles the expenses in one statement that also nulls `trip_id` and `trip_stop_id`, so it can never break the trip-XOR-group CHECK partway: `count` → `is_special=false, trip_id=null, trip_stop_id=null`; `keep` → `trip_id=null, trip_stop_id=null`; `restore` (only with a surviving `source_group_id`) → `special_group_id=source, is_special=true, trip_id=null, trip_stop_id=null`. That is all of the trip's expenses, including ones added after conversion, and the confirm says how many. Only after that succeeds: unarchive the group (restore), then delete the trip. `GET /:id` returns `sourceGroup: { id, name }` (decoded) for the dialog label.
`GET /api/trips/:id/untagged?q=`	Two lists: `during` (inside the trip dates) and `before` (up to 180 days before the start). Expenses with no trip and no special group, read by date window, decoded, then filtered by `q`.

Endpoint	Does
`POST /api/trips/:id/transactions`	`{ transactionIds[] (≤100) }`. One conditional update sets `trip_id`, plus `is_special: true` only for a keep-out trip: `.in('id', ids).eq('user_id').eq('type','expense').is('trip_id', null).is('special_group_id', null).select('id')`. No city writes, because cities are derived. A single statement, so it either applies or it doesn't, and a retry is safe.
`POST /api/trips/from-group`	`{ groupId, trip: <POST body> }`. Creates the trip with `source_group_id`. Then one update moves the group's expenses: `trip_id` set, `special_group_id` null, `is_special` = the switch. Then it archives the group. If the move fails, the trip is deleted and the expenses are untouched (the move was a single statement). If only the archive fails, it is logged; the group just shows as empty and hidden.

Changed: **`POST`** / **`PATCH /api/transactions`**. - Add `tripId` and `tripStopId` (uuid, nullable) to both Zod schemas, `TX_LIST_COLUMNS`, `TX_WRITE_COLUMNS`, the POST insert object and the PATCH update payload. PATCH's existing-row read also selects `trip_id` and `trip_stop_id`. The 2026-08-08 bug was a validated field that was never written; a route test asserts the stored row carries `trip_id` after a POST and after a PATCH. - Rules are checked on the resulting state (`effectiveTrip`, `effectiveStop`, `effectiveType`): - an expense only; - the trip must be the user's; - an override stop must belong to the resulting trip; - a stop requires a trip. - Setting a trip without sending `specialGroupId` → the server clears `special_group_id`. A resulting state with both a trip and a group (both sent, or a group sent onto a row that already has a trip) → 400, never the database CHECK's 500. Mirrored in devMock. - The star is derived only when **`trip_id`** actually changes against the stored row, and only when `isSpecial` is not sent: - joining a keep-out trip → `is_special = true`; - joining a keep-out-off trip → the star is left as it is (the same rule as bulk-adding, so every way of joining gives the same result); - leaving a keep-out trip (cleared, or moved to a keep-out-off trip) → `is_special = false`.

A request that re-sends the same `tripId` never touches the star. An explicit `isSpecial` (the row star, or a deliberate change in the Edit dialog) always wins. The derived star feeds `effectiveSpecial` before the existing group guard runs, so "a group needs a special expense" is judged on the final state. Leaving a keep-out trip while adding a group, without `isSpecial`, is a 400, never an unstarred grouped row. - Changing `tripId` without a `tripStopId` clears the override stop. - Override normalisation, server-side: a `tripStopId` equal to `stopForDate(stops, effectiveDate)` is stored as null. It is just the derived city, and storing it would freeze it, so a later correction of the stop's dates would not move the expense and the row would wrongly show "[pin icon] set by you". Clients also send `tripStopId` only when Alex picked a stop different from the derived one. - `ownsTrip` / `stopBelongsToTrip` helpers follow `ownsSpecialGroup`.

Changed: **`GET /api/dashboard`**. Adds `month.specialOnTripsThisMonth`: the sum of this month's `is_special` expenses with `trip_id` not null, using the same month bounds and the same row set as `specialThisMonth`, so the panel line always reconciles with the hero chip. Mirrored in devMock and tested.

Changed: Phase 17 **`POST /api/subscriptions/:key/attach`** and **`POST /api/subscriptions/track`**. Linking a trip-tagged charge to a subscription, by either route, also takes it off the trip and un-stars it if that trip was keep-out. A subscription is home spending, and the two routes must agree. - attach (one known transaction): one update sets `recurrence_id`, nulls `trip_id` and `trip_stop_id`, and sets `is_special = false` if its trip was keep-out. - /track (several rows): before linking, read the trip of each included id (scoped) and note the keep-out ones. The link update also nulls `trip_id` and `trip_stop_id`. A second update un-stars only the returned ids that were on keep-out trips. If that second update fails, the response is still 201, with `unstarFailed: n`, and the toast says "Tracked — but n charges are still starred; un-star them in Transactions". - The Track confirm mentions the trip removal when any included row is trip-tagged. - Mirrored in devMock and covered in the subscriptions suite.

Changed: timezone report (client **`App.jsx`**). After the zone PATCH succeeds, invalidate `['trips']` and `['dashboard']` (today only `['me']`), so on arrival day the trip card and the Quick Add tag agree on whether the trip has started.

Client

File	Change
`App.jsx`	Nav item Trips between Budgets and Savings. The desktop nav moves from **`md`** to **`lg`** (`hidden lg:flex`, scrolling pills below `lg`). It already overflows between about 768 and 930px today, clipping the theme and logout buttons, and Trips adds about 60px. Checked at 375, 768 and 1024px before done. Prefetch `['trips']` at start. Timezone invalidation as above.
`main.jsx`	Routes `/trips` and `/trips/:id`.
`pages/Trips.jsx`, `pages/TripDetail.jsx`	As described above, each with `QuickAddButton`. TripDetail passes a `presetTripId`.
`components/TripEditor.jsx`	One-save editor (the shared Dialog caps at 92dvh with scrolling content).
`components/TripTag.jsx`, `components/TripPicker.jsx`, `components/TripCard.jsx`	Quick Add tag and chooser; the details/Edit picker; the Dashboard card.
`components/TransactionEditDialog.jsx`	Extracted from `pages/Transactions.jsx` with its save mutation and Phase 17's Subscription section. Used by Transactions and TripDetail.
`components/CategoryManager.jsx`	Export `CategoryDialog` with `lockType` and `onCreated` props.
`components/QuickAddButton.jsx`	Forward `presetTripId` to the dialog.

File	Change
server/routes/dashboard.js	Add trip_id to DASH_TX_COLUMNS (needed for specialOnTripsThisMonth).
server/lib/fx.js	A server copy of QUOTABLE_CURRENCIES for the stop-currency check, with a test that reads client/src/lib/fx.js and asserts the two lists are identical.
components/QuickAddDialog.jsx	The seeding effect and touched-flags; the tag and the trip-dependent omission of isSpecial / specialGroupId in both handleCategoryTap and handleSimpleLog; the tag hidden for income; presetTripId; the "Not trip spending" toast action; the stop-currency seeding and rate memory. The "type it instead" parser keeps the seeded currency, which is visible on the currency chip. The parser cannot tell an explicit currency from its default and does not know EUR, so parser currency handling for trips is out of scope. Post-log toast order: the main toast, the shield toast, then Phase 17's subscription prompt last.
components/SpecialGroupsPanel.jsx, Budgets page, PaceLine.jsx, Settings	The lines described in "Elsewhere".
lib/trips.js	The tripForDate / stopForDate / cityOf mirror and labels.
lib/progressTone.js	progressTone extracted from pages/Budgets.jsx and shared with the trip bars (a shared tone, not a fourth copy). Budgets' private BudgetCard is not reused: it has a weekly-limit display bug, filed separately.
lib/invalidate.js	'trips' added to MONEY_KEYS. Query keys are fixed as ['trips'], ['trips', id] and ['trips', id, 'untagged'], so prefix invalidation reaches all of them.

devMock

Mirror everything in the same session (BUGS-FIXED shape #2): the eight trips endpoints with the same guards and messages, the transactions rules (server-derived star, group clearing, override stop), the Phase 17 attach and /track changes, specialOnTripsThisMonth, and from-group and delete-restore. Import **lib/trips.js**; never re-implement it. Seed one active trip around the mock's "today" with: - two stops, one in euros; - an overall budget, two category budgets and a stop budget; - tagged expenses: a pre-trip flight, a foreign one, and one with an explicit stop override.

Also seed one special group with expenses to convert. While here, fix the existing mock bug where GET /api/transactions?month= passes a Date where monthBounds expects a zone, so the

month filter always shows the current month. Record it in BUGS-FIXED.

Tests

Prerequisite: Phase 17's harness changes (data:null without `.select()`, UUID insert ids, `count`, error injection). The trips suite uses separate child queries, not embedded selects.

- `test/trips.test.js` (pure):
  - `tripStatus` boundaries (start day, end day, a single-day trip, upcoming and past);
  - the total order (equal starts, a day trip inside a stop, archived excluded);
  - `cityOf` (an override beats the date; a pre-trip date gives null);
  - `summariseTrip`: city totals add up to `spent`; other categories; no city; over budget → `overBy`; `perDayLeft` floored and only while active; no overall budget.
- The trips route suite at all three encryption phases, in the style of `affordProjGroupsRoute–Suite.js`:
  - CRUD; ownership 404s for trip, stop, category and group; Zod 400s;
  - postponing a trip with its stops in one PATCH; a category-budget swap; a duplicate category → 409; stops outside the dates → 400;
  - removed-stop overrides nulled; the keep-out flip re-applies, an unchanged value leaves stars alone, and a retry after an injected transactions failure re-applies;
  - setting auto-enable;
  - DELETE with `count`, `keep` and `restore` (restore leaves every row grouped, starred and trip-free; asserted on real PostgreSQL too, since the fake has no CHECKs);
  - a category-budget swap by `category_id`;
  - GET list paging past an injected `maxRows`, with past-trip results present;
  - from-group (the move, stars from the switch, the archive, an injected move failure leaves the group intact);
  - bulk tagging (other users', income, tagged and grouped rows skipped; an existing star kept when keep-out is off; the 100-id cap);
  - `untagged` before and during;
  - stored columns per phase, and every encrypted field written on a rename;
  - the `clean()` leak check.
- The transactions suite:
  - a 3-tap-shaped POST with a keep-out **`tripId`** and no **`isSpecial`** stores **`is_special = true`** (the blocker's regression test);
  - an explicit `isSpecial` wins; leaving a keep-out trip un-stars; re-sending the same **`tripId`** leaves the star alone; joining a keep-out-off trip keeps an existing star; setting a trip clears a group; a resulting trip-plus-group → 400; leaving a keep-out trip plus a group and no `isSpecial` → 400; a `tripStopId` equal to the derived stop is stored as null;
  - stored `trip_id` after POST and PATCH; changing trip clears the override stop; an override stop from another trip → 400; income → 400; another user's trip → 404.
- The subscriptions suite (Phase 17's attach and /track): trip-tagged charges come off the trip; only rows from keep-out trips are un-starred; an injected un-star failure gives 201 with `unstarFailed`.
- `encodeWrite` throws on an array (codec test). `routeMountTable.test.js` lists `/api/trips`. `encryptionScope.test.js` passes with the registry, 018, 018a and 019 in agreement.

- Migration test (opt-in real PostgreSQL). Setup first: the `anon`, `authenticated` and `service_role` roles, `create schema auth`, `auth.users(id uuid primary key)`, an `auth.uid()` stub, and 001 onward against those stubs. Then apply through 017b, then 017b again. Assert:
  - the re-run succeeds;
  - reversed date ranges are rejected; the unique budget per category holds;
  - `trip_xor_group` and `trip_expense_only` reject violations;
  - deleting a trip or stop sets null on transactions (never cascades).
- `npm test` stays green, with the count of real-PostgreSQL tests that actually ran reported.

Manual definition-of-done checks (on top of the build and reachability): - Log an expense through Quick Add's normal 3-tap path inside a keep-out trip, then confirm `/api/budgets` is unchanged and the row is starred. - Log two expenses in a row from the same page inside a keep-out trip with a euro stop: both are tagged and in EUR. Repeat on a device with no remembered rate (day 1). - Fix a typo on a hand-un-starred trip expense via Edit: its star doesn't change. - × one entry, reopen Quick Add: the next entry is tagged again. - Force `/api/fx` to return `rate: null` after typing an amount: the currency does not switch. - Postpone a trip with stops in one save. - Nav at 375, 768 and 1024px.

Rollout (order matters)

1. Read-only live check: still no `*_enc` column and no barrier function. Then Alex runs `017b_trips.sql` in the Supabase SQL editor (exact steps in the hand-off; 017a already applied). A read-only check then confirms the tables and the two CHECKs exist. The API must not deploy first, or every transaction create 500s on the new columns (the Phase 12 lesson).
2. Deploy `trim-api`, then the client.
3. Check: `/api/health` 200; /trips loads; create a test trip; Quick Add shows the tag inside its dates; the monthly budget is unchanged after a 3-tap keep-out expense.

Out of scope (v1)

Ask Trim and "Can I afford this?" answering against the trip; budgets in a local currency; VND stops (needs a manual per-stop rate, the most useful follow-up); income and refunds, including friends paying back a shared dinner; moving expenses between trips in bulk; a Transactions-page trip filter; shared trips; a trips entry in the Wins feed.

Invariants this must not break

- `transactions.amount` stays single-currency.
- Special expenses leave budget math in exactly one place (`lib/special.js`); trips only set the star.
- Quick Add's default path stays 3 taps, nothing it seeds is invisible, and nothing it sends contradicts what is visible (no hidden `isSpecial` or group).
- Every client-supplied id is ownership-checked. A validated field is also written, and mirrored in devMock.
- Guards read decoded rows; responses never return raw decoded rows.
- Deleting a trip or stop never deletes spending.

Known limits (accepted)

- If Alex switches the Special-expenses setting off, keep-out trips stop keeping spending out; a warning line covers this.
- Flipping the keep-out switch resets hand-set stars on that trip (the confirm says so).
- The client mirror of the date rules can’t be unit-tested in the client. It is a few lines with the total order stated beside it.
- A city chosen by hand stays chosen even if its date later moves outside that stop. That is what “set by you” means, and the row shows the [pin icon] marker.